DMLScope: Uncovering Logical Bugs in Database DML via Approximate State-Relation–Driven Metamorphic Testing

Abstract

...

1 Introduction

Database Management Systems (DBMSs) are a fundamental component of modern data-driven applications, ranging from e-commerce and online banking to large-scale enterprise systems [3, 16]. They provide standardized interfaces, such as SQL, to support both data retrieval and data modification [1]. Given their critical role, bugs in DBMSs can lead to severe consequences, including incorrect query results, data corruption, and even security violations [6, 15]. Among various types of bugs, logical bugs are particularly critical because they do not crash the system, but silently lead to incorrect query results or incorrect database states, often remaining unnoticed for long periods [8, 9, 12]. Recent studies have demonstrated that even well-tested DBMSs still contain numerous previously unknown logical bugs, highlighting the difficulty of ensuring their correctness [2, 4, 7, 9–14].

While existing research has made significant progress in detecting logical bugs in read-only queries [2, 4, 7, 9–14], considerably less attention has been paid to state-modifying SQL statements. In this work, we focus on data manipulation language (DML) statements that directly change persistent database contents, including common operations such as `INSERT`, `UPDATE`, and `DELETE`, as well as DBMS-specific extensions such as `REPLACE` when supported. Unlike read-only queries that only return result sets, these statements produce persistent state changes, making their correctness harder to validate and their consequences more severe. For example, an incorrect `INSERT` may introduce spurious tuples, an erroneous `UPDATE` may propagate wrong values to a large number of records, and an unintended `DELETE` may remove data that should have been preserved, leading to irreversible inconsistencies or data loss. Testing such statements is challenging because their semantics are often intertwined with complex predicates, nested subqueries, joins, constraint enforcement, and integrity-related side effects such as cascades. As a result, logical bugs in DBMSs may silently affect which tuples are inserted, modified, or deleted, thereby producing incorrect database states without any explicit failure signal [12].

Logical bugs in DBMSs, especially those in DML statements, are difficult to detect automatically. A fundamental challenge lies in the *test oracle problem* [5], i.e., determining whether the execution of a given SQL statement produces a correct result or database state. While this problem has been extensively studied for read-only queries, it becomes significantly more challenging for DML statements, as correctness must be defined over state transitions rather than query outputs. To address this challenge, recent work DQE [12] proposes a differential testing approach that detects logical bugs by comparing the rows accessed by `SELECT`, `UPDATE`, and `DELETE` statements sharing the same predicate. The key insight of DQE is that these statements should access the same set of tuples under identical conditions, and any inconsistency indicates a potential logical bug. Although effective, DQE relies on cross-statement consistency and treats DML correctness indirectly through row-access equivalence. This design introduces several limitations: (1) it requires restricting SQL features to those commonly supported across different statement types, limiting its applicability to complex queries; (2) it cannot naturally capture value-level correctness (e.g., incorrect assignments in `UPDATE`); and (3) it may fail to detect bugs when multiple statements share the same erroneous execution path [4]. These limitations suggest that a more direct oracle over database state transitions is needed for DML testing.

There have also been several studies on detecting logical bugs in DBMSs [2, 4, 7, 9–14]. For instance, TLP [10] partitions a query based on three-valued logic and checks equivalence between decomposed queries, while NoREC [9] rewrites queries into non-optimized forms to expose inconsistencies. These techniques are effective at detecting logical bugs in *read-only queries*, where correctness can be defined over result sets. However, their primary focus is on query equivalence or result-set relations, rather than the correctness of *state-modifying semantics*. In contrast, our work focuses on logical bugs in DML statements, where errors manifest as incorrect database state transitions, including wrong tuple selection, unintended modifications, and incorrect value assignments. Such bugs occur during the execution of state-modifying operations and cannot be effectively captured by existing query-centric testing approaches.

Inspired by Pinolo [4], which demonstrates that approximate semantic relations can serve as effective invariants for logical-bug detection, we revisit this principle in the context of state-changing SQL. We term this paradigm **Approximate State-Relation–Driven Metamorphic Testing** (ASRMT): for a statement and its over-/under-approximate variant, correctness is validated by whether their post-execution effects satisfy directional partial-order constraints over database state changes (e.g., subset/superset), rather than output equivalence. Any violation of these approximate state-relation invariants indicates a potential logical bug.

2 Background and Motivation

3 Approach

4 Implementation

5 Evaluation

6 Related Work

7 Conclusion

References

[1] Donald D Chamberlin and Raymond F Boyce. 1974. SEQUEL: A structured English query language. In *Proceedings of the 1974 ACM SIGFIDET (now SIGMOD) workshop on Data description, access and control*. 249–264.

[2] Wenqian Deng, Jie Liang, Zhiyong Wu, Jingzhou Fu, and Yu Jiang. 2025. Detecting Logic Bugs in DBMSs via Equivalent Data Construction. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–25.

[3] Daniela Florescu, Alon Levy, and Alberto Mendelzon. 1998. Database techniques for the World-Wide Web: A survey. *ACM Sigmod Record* 27, 3 (1998), 59–74.

[4] Zongyin Hao, Quanfeng Huang, Chengpeng Wang, Jianfeng Wang, Yushan Zhang, Rongxin Wu, and Charles Zhang. 2023. Pinolo: Detecting logical bugs in database management systems with approximate query synthesis. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 345–358.

[5] William E Howden. 2006. Theoretical and empirical studies of program testing. *IEEE Transactions on Software Engineering* 4 (2006), 293–298.

[6] Zu-Ming Jiang, Jia-Ju Bai, and Zhendong Su. 2023. {DynSQL}: Stateful fuzzing for database management systems with complex and valid {SQL} query generation. In *32nd USENIX Security Symposium (USENIX Security 23)*. 4949–4965.

[7] Zu-Ming Jiang and Zhendong Su. 2024. Detecting logic bugs in database engines via equivalent expression transformation. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 821–835.

[8] Matteo Kamm, Manuel Rigger, Chengyu Zhang, and Zhendong Su. 2023. Testing graph database engines via query partitioning. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 23)*. 140–149.

[9] Manuel Rigger and Zhendong Su. 2020. Detecting optimization bugs in database engines via non-optimizing reference engine construction. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1140–1152.

[10] Manuel Rigger and Zhendong Su. 2020. Finding bugs in database systems via query partitioning. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–30.

[11] Manuel Rigger and Zhendong Su. 2020. Testing database engines via pivoted query synthesis. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 667–682.

[12] Jiansen Song, Wensheng Dou, Ziyu Cui, Qianwang Dai, Wei Wang, Jun Wei, Hua Zhong, and Tao Huang. 2023. Testing database systems via differential query execution. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE 23)*. IEEE, 2072–2084.

[13] Jiansen Song, Wensheng Dou, Yu Gao, Ziyu Cui, Yingying Zheng, Dong Wang, Wei Wang, Jun Wei, and Tao Huang. 2024. Detecting Metadata-Related Logic Bugs in Database Systems via Raw Database Construction. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1884–1897.

[14] Xiu Tang, Sai Wu, Dongxiang Zhang, Feifei Li, and Gang Chen. 2023. Detecting logic bugs of join optimizations in dbms. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–26.

[15] Rui Zhong, Yongheng Chen, Hong Hu, Hangfan Zhang, Wenke Lee, and Dinghao Wu. 2020. Squirrel: Testing database management systems with language validity and coverage feedback. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 955–970.

[16] Xuanhe Zhou, Chengliang Chai, Guoliang Li, and Ji Sun. 2020. Database meets artificial intelligence: A survey. *IEEE Transactions on Knowledge and Data Engineering* 34, 3 (2020), 1096–1116.